DATA STARTUP · PROYECTO INTEGRADOR

Entregable E3: Inteligencia en Producción · El Modelo Vive en la Nube

Sistema RentMetrics v3.0 · Arquitectura de Inferencia Desacoplada, Autoescalado y MLOps

Institución: Instituto Tecnológico de Colima (TecNM)

Fecha: Noviembre 2026

Materias: Cómputo en la Nube (IAD-2408) & Analítica de Datos (IAD-2403)

Entregable: E3 (100 pts Nube + 100 pts Analítica)

Equipo: Manuel Alexander Hernández Chávez (Data Lead) · Luis Antonio Hinojosa Corona (Cloud Lead)

Tag Oficial Git: e3 (Rama main)

1. Resumen Ejecutivo del Entregable E3

El Entregable E3 materializa la transición completa del proyecto integrador hacia un entorno de **MLOps en producción**. Los modelos analíticos desarrollados en fases previas han sido serializados, empaquetados en microservicios independientes y orquestados bajo contenedores en la nube con escalabilidad horizontal automática:

- **Modelo SARIMA (1,1,1)x(1,0,1,7):** Proyección del flujo diario con estacionalidad semanal a horizontes de 7 a 30 días, con intervalos de confianza del 80% y 95%. Serializado en models/sarima_model.pkl (Holdout RMSE: $17,334.07, MAPE: 22.49%).
- **Clasificador de Riesgo de Inquilinos:** Evaluación de 4 algoritmos (Gaussian Naive Bayes, KNN, SVM y Random Forest). Selección basada en **Recall (100% de detección de morosidad)**, protegiendo al portafolio contra pérdidas críticas ($> $45,000 MXN). Serializado en models/clasificador_riesgo.pkl.
- **API de Inferencia (FastAPI):** Microservicio dedicado en puerto 8001 con endpoints validados /api/v1/pronostico, /api/v1/clasificar y monitoreo de salud /health.
- **Infraestructura Cloud de Alta Disponibilidad:** 4 contenedores Docker multi-stage con docker-compose.yml, manifiestos completos de Kubernetes (Deployment, Service, HPA 2-8 réplicas), función Serverless event-driven (S3 y SQS con DLQ), e infraestructura declarativa con Terraform.
- **Prueba de Carga con 50 Usuarios Concurrentes (Locust):** 500 peticiones procesadas a **44.63 RPS** con **0.00% de fallos**, latencia p50 de 509 ms, p95 de 4161 ms y disparo verificado del autoescalador de 2 a 5 pods.
- **Tablero v3 en Streamlit:** Interfaz interactiva de analítica avanzada con gráfico Fan Chart y semáforos de riesgo predictivo en tiempo real.

2. Componente de Cómputo en la Nube (Rúbrica: 100 Puntos)

2.1. Contenedores Multi-Stage y Docker Compose (15 Pts)

Se implementaron 4 archivos Dockerfile multi-stage (builder/runtime) minimizando el peso de las imágenes y garantizando seguridad al ejecutar con el usuario no privilegiado appuser:10001:

Servicio	Dockerfile	Puerto	Función en la Solución
Ingesta API	docker/Dockerfile.ingesta	8000	Recepción de transacciones y conciliación en lote
Model Serving API	docker/Dockerfile.modelos	8001	Inferencia SARIMA y clasificador de riesgo en tiempo real
Tablero v3	docker/Dockerfile.tablero	8501	Dashboard ejecutivo en Streamlit con visualizaciones dinámicas

Worker Asíncrono	docker/Dockerfile.worker	-	Procesamiento en background, limpieza de anomalías e inserción SQL

Comando de arranque unificado: docker-compose up -d --build (4 servicios en red rentmetrics-net).

2.2. Despliegue Administrado y Monitoreo de Salud (20 Pts)

El microservicio de modelos expone una sonda activa en /health que valida el estado operativo del servidor y la disponibilidad de los modelos binarios serializados. La solución se encuentra accesible públicamente en Streamlit Cloud: https://luishinojosacorona123-spec-taacos-taacos-nuiqdn.streamlit.app/.

2.3. Orquestación con Kubernetes (15 Pts)

En la carpeta k8s/ se definieron manifiestos declarativos de nivel de producción bajo el namespace rentmetrics: namespace.yaml, configmap.yaml, secret.yaml, ingesta.yaml (Deployment, Service, HPA 2-10 réplicas), modelos.yaml (Deployment con límites de recursos, Service 8001, HPA 2-8 réplicas) y tablero.yaml (Service LoadBalancer 8501).

2.4. Arquitectura Serverless Event-Driven (15 Pts)

El módulo services/serverless/lambda_function.py implementa un manejador dual compatible con AWS Lambda y Cloud Functions. Reacciona reactivamente a eventos de almacenamiento (S3 ObjectCreated en raw/) y colas de mensajería (SQS Records), ejecuta la rutina de limpieza de 5 anomalías con costo de cómputo efímero ($0 en reposo) y canaliza excepciones irrecuperables a la Dead Letter Queue (DLQ).

2.5. Prueba de Carga y Autoescalado con 50 Usuarios Concurrentes (20 Pts)

Se ejecutó una prueba de estrés masiva simulando 50 usuarios concurrentes mediante tests/load/locustfile.py y el benchmark automatizado scripts/ejecutar_prueba_carga.py. Resultados de telemetría oficial (reporte JSON):

Métrica Evaluada	Valor Registrado	Criterio de Rúbrica / SLA	Estado
Usuarios Concurrentes	**50 usuarios**	50 usuarios simultáneos	CUMPLIDO
Transacciones Totales	500 peticiones completadas	Muestra estadística representativa	CUMPLIDO
Rendimiento (RPS)	**44.63 req/seg**	Alta capacidad de absorción	CUMPLIDO
Latencia Mediana (p50)	**509.38 ms**	< 1,000 ms	CUMPLIDO
Latencia Percentil 95 (p95)	**4,161.27 ms**	Acotado bajo estrés máximo	CUMPLIDO
Tasa de Fallos (Error Rate)	**0.00% (0 errores)**	0.00% tolerancia a fallos	CUMPLIDO
Autoescalado HPA	**Escalado de 2 a 5 pods**	Incremento elástico verificado	CUMPLIDO

2.6. Infraestructura como Código (IaC) con Terraform (15 Pts)

En terraform/ se estructuraron los recursos cloud bajo el principio de mínimo privilegio: main.tf crea el bucket S3 (con prefijos raw/ y clean/), colas SQS estándar y DLQ con política de redrive (máximo 3 reintentos), y roles IAM específicos con permisos restringidos para la API y la función Lambda.

3. Componente de Analítica de Datos (Rúbrica: 100 Puntos)

3.1. Modelo Predictivo SARIMA con Bandas de Confianza (45 Pts)

Se implementó el modelo **SARIMA (1, 1, 1) x (1, 0, 1) con periodicidad semanal s=7** sobre el flujo de ingresos diario consolidado:

- **Justificación de Parámetros:** La prueba ADF confirmó estacionariedad con 1 diferencia regular (d=1, $p < 0.001$). El correlograma PACF mostró corte significativo en lag 1 (p=1) y el ACF amortiguamiento en lag 1 (q=1). El término estacional semanal (P=1, D=0, Q=1, s=7) minimizó el Criterio de Información de Akaike (AIC = 13,599.55).
- **Validación Holdout a 30 Días:** RMSE = $17,334.07 MXN, MAE = $15,682.12 MXN, MAPE = 22.49%.
- **Bandas de Confianza:** Proyecciones a 7, 14, 21 y 30 días calculando intervalos al **80% (planeación base)** y **95% (cobertura de estrés)**. Total proyectado a 14 días: $1,097,171.32 MXN.
- **Serialización:** Binario productivo exportado a models/sarima_model.pkl con metadatos en models/sarima_metadata.json.

3.2. Clasificador Multimodelo de Riesgo de Inquilinos (35 Pts)

Se entrenaron y compararon 4 familias algorítmicas mediante validación cruzada estratificada (5-folds) para detectar tempranamente a inquilinos en riesgo de impago / morosidad:

Modelo Clasificador	Exactitud	Precisión	Recall (Moroso)	F1-Score	CV F1 (5-Folds)
Gaussian Naive Bayes	98.41%	0.9500	**1.0000 (100%)**	0.9744	0.9753
K-Nearest Neighbors (k=5)	98.41%	0.9500	**1.0000 (100%)**	0.9744	0.9443
Support Vector Machine (RBF)	98.41%	0.9500	**1.0000 (100%)**	0.9744	0.9538
Random Forest (100 árboles)	100.00%	1.0000	**1.0000 (100%)**	1.0000	**0.9818**

Justificación Rigurosa de la Métrica Rectora (Recall):

En la administración patrimonial de arrendamientos existe una marcada asimetría de costos de error: un **Falso Negativo (FN)** (clasificar a un inquilino moroso como solvente) ocasiona de 3 a 5 meses de renta perdida y costos judiciales de desalojo ($45,000+ MXN). Por el contrario, un **Falso Positivo (FP)** únicamente genera un recordatorio preventivo o llamada de cortesía de 2 minutos ($0 costo). Por ello, el **Recall (100%)** es la métrica prioritaria seleccionada.

El clasificador calibrado fue serializado en models/clasificador_riesgo.pkl.

3.3. Modelos Servidos y Consumo Dinámico en Tablero v3 (20 Pts)

El microservicio FastAPI (services/modelos_api/main.py) expone los endpoints REST para consumo por la interfaz de Streamlit (taacos.py). La pestaña de analítica predictiva renderiza el abanico estocástico (Fan Chart) con áreas translúcidas al 80% y 95%, e incorpora un calculador instantáneo de perfil de riesgo que despliega el semáforo correspondiente (Bajo, Moderado, Crítico) junto con la prescripción operativa de cobranza.

4. Justificación Técnica de Cómputo: CPU vs. GPU (Media Cuartilla)

Dictamen de Arquitectura de Cómputo:
*Se seleccionó **CPU de propósito general optimizada para cómputo** (instancias tipo compute-optimized x86_64 con vectorización SIMD AVX-512) frente a aceleradores gráficos por GPU dedicada, con base en 3 razones técnicas:*
***1. Naturaleza Estructural de los Datos:** El portafolio de RentMetrics se compone de series de tiempo agregadas (638 días) y tablas estructuradas (250 registros). A diferencia del Deep Learning, los algoritmos SARIMA (estimación MLE en espacio de estados), Gaussian Naive Bayes, KNN y Random Forest ejecutan operaciones secuenciales y bifurcaciones que se ejecutan de forma óptima en los registros y caché L2/L3 del procesador.*
***2. Latencia de Transferencia por Bus PCIe:** Enviar lotes de inferencia tabular hacia la memoria VRAM de una GPU a través del bus PCIe introduce una penalización de 20 a 50 ms por llamada, lo que excede el tiempo total de inferencia en CPU (< 4 ms utilizando aceleración NumPy nativa).*
***3. Eficiencia de Costos y Complejidad Operativa:** Una instancia GPU básica en la nube (ej. AWS g4dn.xlarge con NVIDIA T4) representa un costo superior a **$7,500 MXN mensuales** e incrementa las imágenes Docker a más de 3.5 GB. El despliegue en CPU opera con costo marginal ($0 en tiers gratuitos y clústeres elásticos) y arranca en menos de 2 segundos.*

5. Cobertura de Pruebas Unitarias y Enlaces Oficiales

Se ejecutó la suite de pruebas unitarias con pytest, validando la totalidad de los 10 casos de prueba con 100% de éxito en 7.05 segundos:

Módulo de Prueba	Pruebas Ejecutadas	Resultado
tests/test_limpieza.py	Limpieza de 5 anomalías, integridad de bitácora y política DLQ	**3/3 PASSED**
tests/test_modelos.py	Descomposición aditiva, ADF, ACF/PACF y MCO NumPy vs Scikit-Learn	**3/3 PASSED**
tests/test_e3.py	SARIMA con intervalos 80/95%, Clasificador Recall, Model Serving API y Serverless Lambda	**4/4 PASSED**

Enlaces de Acceso al Proyecto:

- **Repositorio GitHub:** https://github.com/luishinojosacorona123-spec/taacos.git (Rama main, Etiqueta e3)
- **Tablero Productivo en la Nube (Streamlit):** https://luishinojosacorona123-spec-taacos-taacos-nuiqdn.streamlit.app/
- **Documentación Técnica Completa:** docs/E3-modelos-produccion.md